

TelemetriaMqtt v2 — Checkpoint 2

Script mock de publicação MQTT: simulando o ESP32 sem hardware

Projeto de Telemetria FSAE

16 de setembro de 2026

Abstract

Este documento detalha o segundo checkpoint do projeto de reconstrução do sistema de telemetria da equipe: um script em Python que simula o comportamento de publicação MQTT do futuro firmware do ESP32, permitindo desenvolver e testar todo o restante do backend (broker, ingestão, dashboards) antes mesmo de o hardware estar em mãos. O checkpoint fixa também o contrato de dados (tópico MQTT e formato do JSON) que será usado pelo firmware real nos checkpoints finais do projeto.

Contents

1	Contexto e objetivo	1
2	O contrato de dados: tópico e formato	2
2.1	Sinais e frequências	2
2.2	Decisão de tópico único e mensagem combinada	2
3	Estrutura de arquivos criada	3
3.1	signal_generator.py	3
3.2	publisher.py	3
3.3	__main__.py	4
4	Desenvolvimento orientado a testes (TDD)	4
4.1	Testes de signal_generator.py	4
4.2	Testes de publisher.py	5
4.3	Resultado da execução	5
5	Validação manual: rodando de verdade	5
6	Comentários explicativos nos testes	6
7	Regra de processo estabelecida a partir deste checkpoint	6
8	Resumo do que ficou pronto	6
9	Próximos passos	6

1 Contexto e objetivo

Um dos desafios identificados no planejamento do projeto foi a dependência de hardware físico (ESP32, transceiver CAN) e de acesso ao carro da equipe para poder desenvolver e testar o

restante do sistema — broker MQTT, script de ingestão, banco de dados de série temporal e dashboards. Esperar por esses recursos físicos atrasaria todo o desenvolvimento do backend.

A solução adotada (decidida em conversa dedicada durante o levantamento de requisitos, documentada no `SCOPE.md`) foi separar dois problemas diferentes:

1. Validar a camada **física** do CAN (leitura real de quadros do barramento do carro através do transceiver) — isso só pode ser feito com hardware real, e fica propositalmente fora do escopo deste checkpoint.
2. Validar a camada **lógica** do sistema (o que acontece depois que um dado chega via MQTT) — isso pode ser simulado inteiramente em software, sem nenhum hardware.

Este checkpoint resolve o segundo ponto: um script Python, rodando no computador do desenvolvedor, que *finje ser* o ESP32 do ponto de vista do broker MQTT — publicando exatamente o mesmo formato de dados que o firmware real vai publicar no futuro (checkpoints 8 e 9).

2 O contrato de dados: tópico e formato

Antes de escrever qualquer código, foi necessário decidir **como** os dados seriam publicados — essa decisão é o contrato entre o firmware (futuro) e todo o resto do sistema (broker, ingestão, dashboards), e precisa ser definida uma vez só, de forma que o mock e o firmware real sejam intercambiáveis sem exigir mudança em nenhum outro componente.

2.1 Sinais e frequências

O ponto de partida foram os 20 sinais de telemetria do carro (RPM, temperatura do motor, correntes de diversos circuitos do PDM, etc.), já mapeados em uma tabela de payloads CAN definida em checkpoint de planejamento anterior (seção "Plano de payloads" do `SCOPE.md`). Cada sinal tem uma frequência de amostragem própria — 20 Hz para sinais de dinâmica rápida do motor (RPM, acelerador), até 2 Hz para sinais lentos (temperatura, estados on/off).

2.2 Decisão de tópico único e mensagem combinada

Em vez de um tópico MQTT por sinal, ou de publicar cada quadro CAN individualmente conforme ele chega (o que geraria uma frequência de publicação irregular e uma mensagem incompleta a cada vez), foi decidido:

- **Um único tópico:** `telemetria/esp32/data`.
- **Uma mensagem JSON combinada**, contendo os 20 sinais e um timestamp (`ts`, em milissegundos), publicada a uma frequência fixa de **20 Hz** — a frequência do sinal mais rápido da tabela.
- Sinais de frequência mais lenta (10, 5 ou 2 Hz) **repetem o último valor amostrado** dentro da mensagem, em vez de serem interpolados, reproduzindo fielmente a granularidade real com que esses dados vão chegar do carro (um sinal de 2 Hz genuinamente só muda a cada 0,5 segundos — fingir uma transição suave seria uma mentira sobre o dado real).

Essa decisão simplifica o consumo dos dados (qualquer assinante do tópico sempre tem uma visão completa e atualizada de todos os sinais, nunca uma mensagem parcial) e mantém o uso de banda previsível (uma mensagem por tick, sempre no mesmo tamanho aproximado), às custas de publicar valores redundantes para os sinais mais lentos — uma troca considerada aceitável dado o volume de dados envolvido.

3 Estrutura de arquivos criada

Listing 1: Estrutura do diretório backend/

```
1 backend/
2   requirements.txt
3   pytest.ini
4   mock_publisher/
5     __init__.py
6     signal_generator.py
7     publisher.py
8     __main__.py
9   tests/
10    test_signal_generator.py
11    test_publisher.py
```

O código foi dividido deliberadamente em duas responsabilidades separadas: `signal_generator.py` contém **lógica pura** (nenhuma rede, nenhum I/O — só cálculo), enquanto `publisher.py` cuida da **integração** (serialização e publicação). Essa separação é o que torna a lógica de geração de sinais trivialmente testável sem precisar de um broker MQTT de verdade.

3.1 signal_generator.py

Define, para cada um dos 20 sinais, sua frequência de amostragem, faixa de valores plausível e um período de oscilação sintética:

Listing 2: Trecho de `mock_publisher/signal_generator.py`

```
1 SIGNAL_SPECS = {
2     "rpm": {"freq_hz": 20, "kind": "analog",
3            "range": (800.0, 8000.0), "period_s": 6.0},
4     ...
5     "bombaInput": {"freq_hz": 2, "kind": "binary",
6                    "range": (0, 1), "period_s": 4.0},
7     ...
8 }
9
10 def _snap_to_interval(t: float, freq_hz: float) -> float:
11     interval = 1.0 / freq_hz
12     return math.floor(t / interval) * interval
13
14 def snapshot(t: float) -> dict:
15     result = {"ts": int(round(t * 1000))}
16     for name, spec in SIGNAL_SPECS.items():
17         sampled_t = _snap_to_interval(t, spec["freq_hz"])
18         ...
19     return result
```

A função `_snap_to_interval` é o coração da simulação: em vez de calcular o valor de um sinal diretamente a partir do tempo corrido `t`, ela primeiro "arredonda" o tempo para baixo até o último múltiplo do intervalo de amostragem daquele sinal específico. Um sinal de 2 Hz (intervalo de 0,5s) avaliado em `t=0.3` e em `t=0.49` produz exatamente o mesmo valor, porque os dois instantes caem dentro da mesma janela de 0,5s — reproduzindo o efeito de "degrau" (staircase) que o dado real vai ter.

3.2 publisher.py

Listing 3: `mock_publisher/publisher.py`

```

1 MQTT_TOPIC = "telemetria/esp32/data"
2
3 def publish_snapshot(client, t: float, dry_run: bool = False,
4                       sink=print) -> None:
5     payload = json.dumps(snapshot(t))
6     if dry_run:
7         sink(payload)
8     else:
9         client.publish(MQTT_TOPIC, payload)

```

O parâmetro `dry_run` permite rodar o script sem nenhum broker MQTT disponível — relevante porque, neste checkpoint, o broker (checkpoint 3) ainda não existia. Em vez de publicar, a mensagem é passada para uma função `sink` (por padrão, `print`), possibilitando tanto o uso interativo no terminal quanto a captura programática em testes.

3.3 `__main__.py`

Um ponto de entrada de linha de comando (`python -m mock_publisher`), com opções para modo `dry-run`, endereço do broker, credenciais e duração da execução — detalhado na Seção ??, onde uma limitação desse arquivo foi encontrada e corrigida no checkpoint seguinte.

4 Desenvolvimento orientado a testes (TDD)

Seguindo a metodologia definida para o projeto, os testes foram escritos antes da lógica de geração de sinais estar finalizada, cobrindo o comportamento esperado antes de qualquer execução real.

4.1 Testes de `signal_generator.py`

Oito testes, cobrindo tanto a estrutura dos dados quanto o comportamento temporal:

- Teste 1** `test_signal_specs_cover_all_20_channels` — garante que a tabela de especificações tem exatamente os 20 canais esperados, nem mais nem menos.
- Teste 2** `test_snapshot_has_all_channels_plus_timestamp` — confirma que a função `snapshot()` retorna exatamente os 20 campos mais `ts`.
- Teste 3** `test_snapshot_values_are_within_declared_range` — avalia `snapshot()` em vários instantes de tempo e confirma que nenhum valor sai da faixa física declarada (ex: RPM entre 800 e 8000).
- Teste 4** `test_binary_signals_are_always_0_or_1` — confirma que os dois sinais digitais nunca assumem um valor fora de $\{0, 1\}$.
- Teste 5** `test_slow_tier_value_is_held_constant_within_its_own_interval` — confirma o comportamento de "degrau": dois instantes dentro da mesma janela de amostragem de um sinal lento produzem o mesmo valor.
- Teste 6** `test_slow_tier_value_changes_after_crossing_its_interval_boundary` — complementa o teste anterior: ao cruzar a fronteira do intervalo, o valor **precisa** mudar (sem esse teste, uma implementação quebrada que sempre retornasse o mesmo número fixo passaria no teste anterior).
- Teste 7** `test_fast_tier_changes_more_often_than_slow_tier_over_one_second` — checagem cruzada entre um sinal de 20 Hz e um de 2 Hz: em um segundo de tempo simulado, o sinal rápido precisa mostrar muito mais valores distintos que o lento.

Teste 8 `test_timestamp_is_monotonic_ms_integer` — confirma que o campo `ts` é sempre um inteiro em milissegundos (compatível com o que `millis()` produziria no firmware real do ESP32), nunca um número de ponto flutuante.

4.2 Testes de `publisher.py`

Três testes, usando um *cliente MQTT falso* (`unittest.mock.MagicMock`) no lugar de uma conexão de rede real — decisão deliberada, já que neste checkpoint não existia nenhum broker MQTT disponível para testar contra:

Teste 1 `test_publish_snapshot_calls_client_publish_with_correct_topic` — confirma que a função chama `client.publish` exatamente uma vez, com o tópico correto.

Teste 2 `test_publish_snapshot_payload_is_valid_json_with_expected_keys` — confirma que o payload publicado é uma string JSON válida, e que o campo `ts` corresponde corretamente ao tempo passado (ex: `t=2.5` produz `ts=2500`).

Teste 3 `test_publish_snapshot_dry_run_does_not_touch_client` — confirma que, em modo dry-run, o cliente MQTT (mesmo que fosse um objeto real) nunca é tocado — só a função `sink` alternativa é chamada.

4.3 Resultado da execução

Listing 4: Saída real de `pytest -v`

```
1 tests/test_publisher.py::
  test_publish_snapshot_calls_client_publish_with_correct_topic PASSED
2 tests/test_publisher.py::
  test_publish_snapshot_payload_is_valid_json_with_expected_keys PASSED
3 tests/test_publisher.py::test_publish_snapshot_dry_run_does_not_touch_client
  PASSED
4 tests/test_signal_generator.py::test_signal_specs_cover_all_20_channels
  PASSED
5 tests/test_signal_generator.py::test_snapshot_has_all_channels_plus_timestamp
  PASSED
6 tests/test_signal_generator.py::
  test_snapshot_values_are_within_declared_range PASSED
7 tests/test_signal_generator.py::test_binary_signals_are_always_0_or_1 PASSED
8 tests/test_signal_generator.py::
  test_slow_tier_value_is_held_constant_within_its_own_interval PASSED
9 tests/test_signal_generator.py::
  test_slow_tier_value_changes_after_crossing_its_interval_boundary PASSED
10 tests/test_signal_generator.py::
  test_fast_tier_changes_more_often_than_slow_tier_over_one_second PASSED
11 tests/test_signal_generator.py::test_timestamp_is_monotonic_ms_integer PASSED
12
13 11 passed in 0.02s
```

Todos os 11 testes passaram na primeira execução completa, com o conjunto inteiro rodando em 0,02 segundos — consequência direta de `signal_generator.py` não depender de rede, disco ou qualquer recurso externo lento.

5 Validação manual: rodando de verdade

Além dos testes automatizados, o script foi executado de fato em modo dry-run, para observar visualmente o comportamento de "degrau" descrito na Seção 4.1:

Listing 5: Saída real de `python -m mock_publisher -dry-run`

```
1 {"ts": 0, "rpm": 4400.0, ..., "engineTemp": 90.0, "airTemp": 30.0, ...}
2 {"ts": 50, "rpm": 4588.41, ..., "engineTemp": 90.0, "airTemp": 30.0, ...}
3 {"ts": 100, "rpm": 4776.3, ..., "engineTemp": 90.0, "airTemp": 30.0, ...}
4 {"ts": 150, "rpm": 4963.16, ..., "engineTemp": 90.0, "airTemp": 30.0, ...}
5 {"ts": 201, "rpm": 5148.48, "gpsSpeed": 95.65, ..., "engineTemp": 90.0, ...}
```

O comportamento observado confirma exatamente o design: o campo `rpm` (20 Hz) muda a cada mensagem (a cada 50 ms); o campo `gpsSpeed` (10 Hz) só muda a cada duas mensagens; e o campo `engineTemp` (2 Hz) permanece constante em 90.0 por toda a janela observada, exatamente como o sinal equivalente vai se comportar quando vier do carro real.

6 Comentários explicativos nos testes

Após a primeira versão dos testes, foi solicitada uma revisão com comentários explicando o propósito de cada teste individualmente — não apenas o nome da função, mas o raciocínio por trás dela. Cada teste em `test_signal_generator.py` e `test_publisher.py` recebeu um comentário curto explicando o que ele verifica e, quando relevante, por que ele existe separado de outro teste semelhante (como o par "valor congelado dentro do intervalo" / "muda ao cruzar o intervalo", onde um teste sozinho não seria suficiente para pegar uma implementação incorreta).

7 Regra de processo estabelecida a partir deste checkpoint

Durante a revisão deste checkpoint, foi estabelecida uma regra de processo que passou a valer para todo o restante do projeto (registrada em `CLAUDE.md`): **testes, uma vez escritos, são intocáveis**. Nunca devem ser apagados ou modificados para "fazer passar" — se um teste falha, o problema está na implementação, não no teste. Essa regra existe porque é exatamente o que torna a abordagem de TDD do projeto uma garantia real de qualidade, em vez de uma formalidade contornável.

8 Resumo do que ficou pronto

- Contrato de dados definido e documentado: tópico único, mensagem JSON combinada a 20 Hz, com "degrau" nos sinais mais lentos.
- Script mock completo (`backend/mock_publisher/`), capaz de rodar tanto em modo dry-run (sem broker) quanto conectado a um broker MQTT real.
- 11 testes automatizados, cobrindo tanto a lógica de geração de sinais quanto a integração de publicação.
- Validação manual confirmando visualmente o comportamento de amostragem por camadas de frequência.
- Regra de processo estabelecida: testes nunca são apagados ou alterados para acomodar uma implementação incorreta.

9 Próximos passos

O checkpoint seguinte (checkpoint 3, já concluído e documentado separadamente) coloca um broker MQTT real no ar via Docker, permitindo que este script mock finalmente publique para um destino de verdade, em vez de apenas para a tela do terminal.